

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 07 -

BOAS PRÁTICAS E AUTOMAÇÃO (CI/CD) EM MLOPS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	24
O QUE VOCÊ VIU NESTA AULA?	27
REFERÊNCIAS	28

EMENDAS

O QUE VEM POR AÍ?

Nos bastidores de sistemas de Machine Learning de sucesso existe um conjunto de práticas de engenharia que garante que modelos recém-criados em laboratório cheguem ao ambiente de produção de forma confiável, reproduzível e contínua. Nesta aula, você descobrirá como os princípios de Integração Contínua (Continuous Integration, CI) e Entrega Contínua (Continuous Delivery, CD) se aplicam ao fluxo de Machine Learning – uma disciplina conhecida como MLOps (Machine Learning Operations). Iremos “quebrar o gelo” explorando os desafios que surgem quando tentamos levar modelos de ML do experimento à produção e como a automação desses processos ajuda a superar obstáculos como falhas na reprodutibilidade e implantações arriscadas. Imagine treinar um modelo de alta acurácia em seu notebook, mas ao implantá-lo manualmente enfrentar erros inesperados devido a diferenças de ambiente ou dados não previstos – sem as devidas práticas de MLOps, situações assim podem comprometer todo um projeto de IA.

Ao longo desta aula, apresentaremos como estruturar pipelines automatizados de ML alinhados às boas práticas do mercado, integrando ferramentas como Git (para versionamento de código), DVC (Data Version Control, para dados), MLflow (rastreamento de experimentos e modelos) e plataformas de deploy em cloud. Você entenderá como o MLOps amplia o DevOps tradicional: além de código, passamos a integrar dados e modelos no ciclo contínuo de testes e entrega. Vamos engajar este tema mostrando exemplos reais e convidando você a pensar sobre a confiabilidade dos sistemas de IA. Ao final, você terá aprendido por que CI/CD é fundamental em projetos de Machine Learning, obtendo as bases para evoluir de protótipos experimentais para pipelines de ML em produção, de forma escalável e segura.

HANDS ON

A fase prática desta aula simula um pipeline de MLOps automatizado semelhante ao adotado por empresas que operam modelos de ML em produção. Nesta atividade hands on, você irá vivenciar a implementação de CI/CD para Machine Learning, integrando controle de versão, treinamento automatizado e deploy contínuo de modelos. Em especial, vamos utilizar GitHub Actions (para orquestrar as etapas de CI/CD), juntamente com o DVC (para controle de dados e versionamento de modelos) e o MLflow (para registro e gestão de modelos). O objetivo é mostrar, na prática, como um commit de código ou a chegada de novos dados pode disparar automaticamente todo um fluxo: treinamento do modelo, validação de métricas e, se tudo estiver conforme o esperado, implantação do modelo atualizado em um ambiente de produção simulado.

Durante o hands on, estruturaremos o pipeline CI/CD com gatilhos bem definidos e etapas modulares. Por exemplo, configuraremos eventos de disparo (triggers) como um commit no repositório Git – a cada mudança no código do projeto de ML, nossa pipeline será iniciada. Da mesma forma, poderíamos acionar o fluxo quando há atualização de dados (novos dados de treinamento) ou ajustes de hiperparâmetros importantes. O pipeline será separado em estágios claros: uma etapa de treinamento do modelo, seguida de uma etapa de validação, onde testes automatizados verificam se os dados estão corretos e se o modelo atinge as métricas mínimas desejadas. Por fim, a etapa de deploy, que simula a promoção do modelo para uso em produção caso todas as validações sejam aprovadas.

Código completo disponível no GitHub. Repositório de exemplo: <https://github.com/Cataldir/Materiais-MLET/tree/main/fase-02-containers-e-ambientes-reprodutivos/04-dvc-mlflow>.

SAIBA MAIS

Em projetos de Machine Learning, mudanças são constantes – seja no código, nos dados ou nos parâmetros dos modelos – o que torna inviável uma gestão manual e esporádica dessas atualizações. Sem práticas de Integração e Entrega Contínua, equipes de ML enfrentam problemas graves: deploy de modelos não validados (o que pode levar modelos defeituosos ao usuário final), quebra de reprodutibilidade de experimentos (dificultando depuração e auditoria) e falta de rastreabilidade das versões de modelos, resultando em alto risco operacional e técnico.

Não por acaso, estima-se que a maioria dos projetos de ML experimentais falha em chegar a produção de maneira sustentada justamente por não endereçar esses desafios de automação (Salama et al., 2021 apud Eken et al., 2024). O termo MLOps surgiu como resposta a esse cenário, sendo proposto na comunidade de pesquisa por volta de 2015 (Sculley et al., 2015) para acelerar o ciclo de vida de ML com escalabilidade e confiabilidade similares às do DevOps tradicional. Em essência, o MLOps visa aplicar ao pipeline de Machine Learning os mesmos princípios que o DevOps trouxe para o desenvolvimento de software convencional: automação, padronização e monitoramento contínuo em todas as etapas, do desenvolvimento à operação. Apesar de ML systems serem, no fim, software, eles diferem de sistemas tradicionais em vários aspectos fundamentais – e é exatamente nesses pontos que o CI/CD em MLOps precisa focar.

Primeiramente, as equipes tendem a ser multidisciplinares: além de engenheiros de software, há cientistas de dados que podem não ter experiência em práticas de engenharia de produção. O processo de desenvolvimento de modelos é altamente experimental e iterativo, envolvendo tentativa e erro com dados e algoritmos; manter reprodutibilidade nesse contexto é desafiador, pois precisamos versionar não só código, mas também dados de treinamento e configurações de modelo (o que chamamos de “versionamento triplo” – código, dados e modelo versionados em conjunto para cada experimento).

Os testes em sistemas de ML também diferem: além de testes unitários de funções, devemos incluir validação de dados (verificar esquemas, formatos, outliers)

e avaliação de qualidade do modelo. Por exemplo: garantindo que métricas em um conjunto de teste atinjam certo patamar. No deploy, enquanto em software tradicional implantar uma nova versão equivale a atualizar um serviço, em ML muitas vezes precisamos implantar todo um pipeline de retraining automático – integrando novos dados e recalibrando o modelo periodicamente.

Por fim, em ambiente de produção, modelos de ML podem degradar de forma silenciosa conforme os dados reais mudam (fenômeno de data drift ou model decay); portanto, operações de ML exigem monitoramento contínuo não só de logs e latência, mas também do desempenho preditivo do modelo ao longo do tempo. Em resumo, CI/CD em MLOps é essencial porque responde a essas particularidades: ele passa a testar e validar também dados e modelos, não apenas código; orquestra pipelines complexos de treinamento e deploy; e inclui mecanismos de Continuous Training (CT) e monitoramento para reagir à evolução do modelo no mundo real.

CI/CD tradicional vs CI/CD em MLOps

No DevOps tradicional, a integração contínua envolve integrações frequentes de código em um repositório central com testes automatizados e a entrega contínua garante que a aplicação esteja sempre pronta para release. Em MLOps mantemos esses conceitos, mas os ampliamos.

Integração Contínua (CI) em ML não se limita a verificar código e componentes, mas também incorpora testes de dados (ex.: validar se dados novos seguem o esquema esperado, detecção de drift estatístico) e modelos (ex.: verificar se um modelo treinado atende métricas de desempenho). A CI abrange pipelines de dados – por exemplo, executando rotinas de preparação de dados em ambiente de teste para garantir que continuem funcionais com uma nova alteração de código ou de dependências. Além disso, a CI de ML foca na reprodutibilidade: cada commit no repositório de código, quando combinado com uma versão específica dos dados, deve produzir resultados consistentes. Isso leva à prática de orquestrar experimentos automatizados: um sistema de CI pode treinar o modelo com um pequeno subconjunto de dados (teste rápido de pipeline) sempre que o código mudar, garantindo que a integração entre componentes de dados, modelo e código esteja funcionando.

Entrega Contínua (CD) em ML envolve automatizar o deploy não de um binário simples, mas de um conjunto de artefatos: o modelo treinado, seu pipeline de inferência e possivelmente componentes de dados. Técnicas tradicionais de implantação, como blue-green ou canary deployment, precisam ser adaptadas para modelos de ML. Por exemplo, ao lançar uma nova versão de um modelo de recomendação, pode-se fazer um deploy canary onde apenas x% dos usuários recebem as recomendações do novo modelo enquanto o restante continua com o modelo antigo, permitindo comparar métricas online antes de promover totalmente a nova versão.

O CD em ML muitas vezes requer pipelines extras de verificação pós-deploy: monitorar se a distribuição de entrada do modelo em produção está dentro do esperado, caso contrário, acionando alarmes ou revertendo para o modelo anterior. Em outras palavras, a CD para ML assume que o “deliverable” não é apenas código, mas um sistema de ML completo que inclui dados versionados, modelo e configuração – e a entrega contínua deve montar e implantar esse sistema garantindo que todas as peças se encaixem.

Ferramentas específicas, como sistemas de Model Serving, entram em cena para possibilitar atualizações frequentes do modelo em produção sem interromper serviços.

Continuous Training (CT): um conceito adicional em MLOps que não existe no DevOps tradicional é o de treinamento contínuo. Significa ter pipelines automatizados que, periodicamente ou por gatilhos, retreinam o modelo com novos dados e disponibilizam a nova versão de forma integrada. Por exemplo, um modelo de detecção de fraudes pode ser programado para retreinar semanalmente incorporando os novos exemplos de fraude identificados na semana e o pipeline CT cuidará de treinar, validar e implantar automaticamente essa versão. O CT fecha o ciclo contínuo de ML, garantindo que o modelo se atualize em resposta a drift ou novos padrões nos dados.

Enquanto o CI/CD tradicional lida com um artefato estático (aplicação) passando de desenvolvimento para produção, o CI/CD em ML lida com um artefato dinâmico (modelo) que passa continuamente por um ciclo de (dados + código) -> modelagem -> validação -> deploy. Essa diferença exige novas práticas e ferramentas

especializadas, que discutiremos a seguir, mas o objetivo final permanece alinhado: entregas mais rápidas, seguras e confiáveis. Conforme destacado pela Google, muitas das armadilhas de se operar ML em produção (as “cartas de crédito de alto juro do débito técnico”) são sanadas ao adotar esse paradigma de automação contínua, evitando processos manuais sujeitos a erro.

Automação de treinamento

Uma das primeiras fronteiras ao se instituir CI/CD para ML é automatizar o treinamento do modelo e as operações relacionadas. Em um fluxo tradicional, um(a) cientista de dados treina modelos manualmente, executando scripts quando julga necessário. No MLOps, buscamos que esse processo seja padronizado e acionado de forma automática sempre que houver mudanças relevantes.

Isso traz diversos benefícios: reexecução consistente ao mudar dados ou código, padronização de experimentos (todo modelo treinado passa pelos mesmos passos, reduzindo variabilidade), redução de retrabalho manual e escalabilidade – é possível treinar modelos com mais frequência ou em paralelo, conforme demanda. Para automatizar o treinamento, o primeiro passo é codificar o pipeline de ML. Ferramentas como DVC, Kubeflow ou Apache Airflow permitem descrever as etapas do treinamento (preparação de dados, extração de features, treino, avaliação) e suas dependências. Assim, quando novos dados são disponibilizados ou quando o código de alguma etapa muda, o sistema identifica automaticamente que partes do pipeline precisam rodar novamente.

Por exemplo, ao atualizar um módulo de engenharia de atributos, o pipeline detecta que precisa refazer a etapa de extração e depois retreinar e reavaliar o modelo. Essa automatização garante repetibilidade: se hoje e daqui a um mês executarmos o pipeline com o mesmo snapshot de dados e código, os resultados serão os mesmos (salvo interferência de aleatoriedade, que por sinal deve ser controlada via seeds). Recursos como DVC e MLflow atuam juntos para garantir controle e rastreabilidade durante o treino automatizado. O DVC assegura que estamos usando o dataset correto na versão correta, enquanto o MLflow (ou outro tracker) loga parâmetros e métricas de cada execução do treinamento.

Com integração ao CI, a cada commit relevante podemos acionar um treinamento automatizado (usando talvez uma versão reduzida dos dados para agilidade) e registrar os resultados. Isso cria um histórico rico de experimentos, integrado ao controle de versão de código, que documenta a evolução do modelo. Também possibilita testes automatizados no treinamento: por exemplo, podemos inserir no pipeline uma verificação para garantir que o tempo de treinamento não exceda X minutos ou que o modelo converge (não resultando em NaNs nas métricas), falhando o processo se algo der errado. Esses são análogos a testes de integração no software, mas aqui aplicados ao processo de otimização do modelo.

Outra faceta da automação de treinamento é a execução escalável. Em vez de treinar modelos apenas no computador local, pipelines MLOps frequentemente utilizam infraestruturas de computação distribuída ou em nuvem para paralelizar e acelerar o treinamento. Por exemplo, com abordagem declarativa, é possível que o step de treinamento seja embalado em um contêiner Docker e o CI/CD dispare esse contêiner em instâncias EC2 ou em um cluster Kubernetes para treinar com GPUs, fazendo scale out conforme necessidade.

Em empresas com múltiplos times reutilizando recursos, é comum integrar a automação de treino com sistemas de gerenciamento de cluster (como Kubernetes + Kubeflow) para otimizar o uso de recursos (agendando jobs de ML quando há máquinas ociosas, por exemplo). O resultado é que o treinamento do modelo torna-se uma peça rotineira e previsível da cadeia de entrega de software, não mais um evento esporádico e artesanal.

Por fim, a automação de treinamento facilita a adoção de técnicas mais avançadas como AutoML dentro do pipeline. Ferramentas de AutoML (p. ex. AutoKeras, H2O, TPOT) podem ser acionadas automaticamente para buscar melhores hiperparâmetros, ou até arquiteturas de modelo e seus resultados avaliados no pipeline. Isso significa que sem intervenção humana, uma pipeline pode tentar 10 configurações de modelo e escolher a melhor, registrando-a. Essa abordagem, porém, aumenta muito a carga computacional, devendo ser usada com critério.

Em ambiente de CI, seria possível rodar apenas nightly ou em triggers específicos devido ao custo. Mas isso ilustra o poder: ao elevar o nível de automação, abrimos espaço para inovações e otimizações contínuas.

Automação de validação

Nenhum modelo de Machine Learning deveria chegar ao usuário final sem passar por uma bateria de validações – isso inclui validar os dados que alimentam o modelo, as previsões geradas e o próprio desempenho do modelo em critérios de aceitação. Automatizar a validação significa incorporar essas verificações como etapas obrigatórias dentro do pipeline CI/CD, em vez de depender de inspeções manuais pontuais. Essa é uma área crítica onde MLOps se diferencia, pois entendemos que a qualidade do produto de ML não depende só de ausência de bugs no código, mas também de confiabilidade dos dados e robustez estatística dos modelos.

Uma parte da validação automática é a validação de dados (data validation). Antes de treinar ou servir um modelo, precisamos garantir que os dados estão no formato e faixa esperados. Ferramentas como Great Expectations ou TensorFlow Data Validation (TFDV) permitem definir expectativas sobre os dados: por exemplo, “a coluna Idade deve conter apenas valores ≥ 0 e ≤ 120 ” ou “a distribuição das classes na variável alvo não deve mudar drasticamente em relação ao último batch”. Essas validações podem rodar em pipelines de CI tanto em dados de treino (checando que não há corrupção nos novos dados ingeridos) quanto em dados de produção (checando que o input do modelo não desviou muito da distribuição original). Quando uma validação de dados falha – digamos, um campo importante veio vazio ou com tipo errado –, o pipeline acusa erro e nenhum modelo é treinado ou atualizado até que a anomalia seja investigada. Isso previne que dados de baixa qualidade propaguem o erro adiante.

Como exemplo, o sistema de validação de dados do Airbnb (Atd, internal) rodava em cada atualização de dados para seus modelos de busca, bloqueando automaticamente o uso de datasets que não atendessem invariantes predefinidas, o que evitou a implantação de modelos treinados com dados incorretos em várias ocasiões (relato hipotético inspirado em práticas recomendadas na indústria).

Outra camada é a validação de métricas de modelo (já mencionada nas boas práticas). Aqui, após o treinamento, o pipeline executa avaliações quantitativas no modelo – por exemplo, calcula acurácia, precisão, recall, AUC, erro quadrático médio,

dependendo do problema – e compara com critérios. Isso pode incluir comparações com modelos anteriores: por exemplo, “o novo modelo deve ter F1 no mínimo igual ou superior ao modelo atualmente em produção” (para evitar regressão de performance). Em setups avançados, pode-se até usar testes estatísticos de significância para verificar se uma melhora é confiável ou se está dentro da variação aleatória. Tudo isso pode (e deve) ser automático.

Um caso notável é o do sistema de gates do LinkedIn: todo modelo de recommender que os engenheiros queriam lançar passava por uma série de testes offline e online; se não superasse o modelo atual em X de 10 métricas-chave (incluindo métricas de negócio como retenção), o sistema automatizado rejeitava o lançamento, forçando mais iteração. Esse tipo de rigor se torna escalável só com automação. Além das métricas, há validações qualitativas/consistência: por exemplo, garantir que as previsões do modelo não quebrem certas regras de negócio. Suponha um modelo que precifica seguros de carros – podemos inserir uma verificação pós-treino para assegurar que nenhum grupo demográfico receba preço zero ou negativo, ou que um carro mais valioso nunca receba prêmio menor que um carro mais barato (monotonicidade). Tais condições de negócio podem ser codificadas como testes (até usando exemplos sintéticos) e executados automaticamente contra o modelo treinado. Se qualquer condição falhar, o modelo é descartado. Isso incorpora no CI/CD uma forma de teste funcional de modelos, garantindo alinhamento com requisitos além das métricas puras.

Na prática, implementar automação de validação muitas vezes requer orquestrar comparações entre modelos. Ferramentas de Model Registry (como MLflow, SageMaker Model Registry) facilitam isso ao permitir armazenar e recuperar a versão anterior do modelo em produção. Um pipeline automatizado pode pegar o “Champion” (modelo atual) e o “Challenger” (modelo novo treinado) e rodar ambos em um conjunto de teste ou de validação, gerando um relatório comparativo. Essa técnica, chamada de shadow evaluation (o modelo novo é avaliado nas sombras), pode ser integrada antes do deploy: se o Challenger não for consistentemente melhor, a pipeline talvez decida nem promovê-lo. Em alguns cenários, empresas chegam a servir o modelo novo em sombra por um tempo – isto é, ele recebe o mesmo tráfego do modelo atual mas suas respostas não afetam o usuário, apenas são registradas

para análise – e métricas de produção são comparadas. Isso geralmente requer sistemas de produção prontos para lidar com “shadow models”, mas a análise resultante pode ser automatizada e integrada no pipeline de deploy contínuo.

Por exemplo, um modelo de recomendação novo pode ser colocado em sombra por 24h e se suas taxas de clique previstas ficarem abaixo do atual em >5%, o sistema aborta o rollout.

Um ponto importante: a automação de validação não elimina a necessidade de revisão humana, mas a desloca para onde agrega mais valor. Em vez de humanos terem que verificar manualmente cada métrica ou plotar gráficos para decidir se implantam um modelo, eles podem focar nos casos em que a automação sinalizou um problema ou anomalia – por exemplo, se um alerta indicar drift de dados, uma análise humana se aprofunda e decide se é necessário ajustar o pipeline. A automação atua como primeiro filtro incansável e consistente. Além disso, as validações automáticas geram logs e relatórios que servem como documentação, útil para auditorias e compliance. Por exemplo, demonstrar a um órgão regulatório que nenhum modelo de crédito é promovido sem atingir certo nível de equidade, se isso estiver automatizado e registrado, facilita conformidade.

Automação de deploy

Superados os testes e validações, chegamos ao momento de colocar o modelo “no mundo real”. A automação de deploy em MLOps visa tornar a promoção de novos modelos para produção tão simples e segura quanto possíveis deploys de aplicações em um pipeline de DevOps. Isso significa que, uma vez aprovado, o modelo deve ser implantado de forma rápida, reproduzível e com possibilidade de rollback fácil. Em um cenário ideal de CI/CD de ML, quando o pipeline decide promover um modelo, ele automaticamente registra o modelo (se já não estiver registrado) em um Model Registry central, com um novo versionamento.

- **Implanta o modelo em um ambiente de produção** – isso pode assumir várias formas dependendo da arquitetura: por exemplo, criar um novo serviço de inferência em um cluster Kubernetes usando a imagem do

modelo, ou publicar o modelo em um endpoint de nuvem (AWS Sagemaker endpoint, Google AI Platform model etc.), ou inserir o modelo em uma pipeline de streaming (no caso de modelos de stream processing).

- **Reconfigura os consumidores para usar o novo modelo.** Se usamos uma arquitetura de microsserviços, talvez seja algo sutil como atualizar uma variável de ambiente ou registro de serviço para apontar para a nova versão do modelo. Em outros casos, pode envolver atualizar um arquivo de modelo em um storage que é lido por um serviço de predição em batch. O importante é que isso ocorra sem necessidade de manuseio manual.
- **Verifica a saúde após deploy** – por exemplo, automaticamente roda um sanity check: faz algumas inferências de teste com a nova versão no ambiente de produção e verifica se responde corretamente.
- **Promove ou reverte conforme o caso:** se o modelo novo ficar ativo e as métricas de produção se mantiverem boas, consolida-se o deploy; se algo der errado (erros aumentam, performance cai), aciona-se o rollback. Falando assim, parece muito similar ao DevOps – e de fato é, só que aplicado a um artefato diferente (o modelo).

Ferramentas de orquestração de contêineres e funções serverless facilitam bastante essa automação. Por exemplo, se o modelo é empacotado em uma imagem Docker (contendo o código de predição), podemos ter no pipeline um passo de Build de imagem + Deploy em Kubernetes. Com configurações de Kubernetes, podemos implementar estratégias conhecidas: um deploy Blue-Green, onde se levanta a nova versão (green) em paralelo à antiga (blue), direciona-se progressivamente o tráfego para a nova e, se tudo bem, desativa-se a antiga – caso haja problemas, reverte-se instantaneamente ao blue. Essa estratégia minimiza downtime e risco e, com automação, o pipeline pode executar todo o procedimento e até notificar responsáveis “Novo modelo em produção, tráfego 100% direcionado ao v2”. Alternativamente, um deploy canário envia inicialmente, digamos, 5% do tráfego real para o modelo novo e mantém 95% no antigo; o sistema monitora métricas (p. ex. taxa de erro, latência, ou mesmo métricas de negócio se medíveis em tempo real) e, gradualmente, aumenta a fatia do novo modelo se não detectar regressões. Várias plataformas de MLOps

oferecem esse recurso out-of-the-box, integrando com ferramentas de A/B testing ou com intelligent routing.

Em resumo, aplicamos as estratégias já consagradas de deploy contínuo também aos modelos de ML. Uma complicação de ML é que nem sempre “um modelo = um serviço”. Às vezes temos pipelines de predição batch: por exemplo, toda madrugada gerar previsões de cancelamento para todos clientes e salvar no banco. Nesse caso, o deploy do modelo envolve agendar um novo job de batch com o novo modelo ou atualizar um script de workflow.

Ferramentas como Apache Airflow podem tornar isso dinâmico: o pipeline CI/CD, ao receber um modelo novo, atualiza a DAG do Airflow para usar o modelo X versão Y na próxima execução agendada. Esse é um padrão diferente do deploy online, mas é igualmente importante automatizar para evitar confusões (quantas vezes em empresas predições batch ficam desatualizadas porque alguém esqueceu de rodar manualmente com o modelo novo...).

Um componente estreitamente ligado ao deploy automatizado é o Model Registry. Ele atua como a fonte da verdade sobre quais modelos estão em qual estágio (Staging, Production etc.). O pipeline de CI/CD deve interagir com o registry para marcar a promoção do modelo. Por exemplo, no MLflow Registry podemos programar: “ao concluir as validações, marcar a versão 5 do modelo ChurnPrediction como Production e desfazer marca da versão 4 para Archived”. Essas transições podem acionar hooks ou notificações – por exemplo, disparar um evento ou chamada que atualiza o serviço de inferência.

Muitos fluxos de MLOps usam essa abordagem: o serviço de inferência não recebe o modelo binário via CI, mas sim ouve o registry – quando um modelo muda de estágio, o serviço baixa a nova versão automaticamente. Assim, o deploy se efetiva via registro.

Qualquer que seja a arquitetura, o registro facilita ter o histórico: se algo der errado, sabemos qual versão existia antes e podemos facilmente reverter a ela.

Falando em rollback automático, essa é outra característica crucial. Em CI/CD de software, se uma nova versão do app causa alta latência ou erros 500, ferramentas como Kubernetes ou AWS CodeDeploy podem acionar rollback para versão anterior.

Em modelos de ML, podemos definir critérios de rollback baseados em métricas de negócio monitoradas. Por exemplo, suponha que um modelo de recomendação novo foi implantado e notamos nas horas seguintes que a taxa de cliques caiu 15% (um sinal de piora) – um sistema de monitoramento pode, ao detectar essa queda além de um limiar, automaticamente reverter para o modelo anterior. Isso requer que o modelo anterior esteja facilmente disponível (o que é provido pelo registry) e que a arquitetura suporte a troca rápida (no caso de Blue-Green, manter o blue rodando por um tempo para facilitar voltar).

A automação do rollback diminui o tempo de exposição a um modelo problemático, sem depender de alguém ser acordado de madrugada para corrigir. Cabe salientar que, embora automatizado, o deploy de modelos deve respeitar governança e aprovação em ambientes críticos. Em setores regulados (bancos, saúde), às vezes exige-se validação humana ou documentação formal para ativar um modelo. Nesses casos, o pipeline CI/CD pode parar na etapa “modelo aprovado, pronto para deploy” e solicitar um approval manual (por exemplo, via uma interface, um gerente clica autorizando) para então seguir e implantar. Ou pode implantar em staging e esperar validação manual para promover a produção. Mesmo com esse gargalo deliberado, todo o trabalho braçal de preparo do modelo já foi feito pela automação – a intervenção humana entra apenas como controle de qualidade final.

Em termos de velocidade, empresas líderes alcançaram ciclos de deploy impressionantes graças a MLOps. O Uber, por exemplo, com sua plataforma Michelangelo, reportou que um novo modelo pode ser treinado, validado e disponibilizado em seus microsserviços globais em questão de horas, algo impensável sem automação de ponta a ponta. Já o Google implementou internamente sistemas de deploy contínuo de modelos que permitiram atualizações semanais de modelos de tradução do Google Translate, com monitoramento intensivo para garantir que cada atualização fosse um passo incremental positivo (exemplo hipotético para ilustração).

Esses casos reforçam: a automação do deploy de modelos colhe os frutos das etapas anteriores – se treinamos e validamos automaticamente, entregar automaticamente fecha o ciclo e gera um fluxo constante de melhorias chegando ao usuário.

Governança e compliance

À medida que empresas tornam decisões de Machine Learning parte central de seus produtos, aspectos de governança, auditoria e compliance se tornam cruciais. Quem aprovou o modelo? Com quais dados ele foi treinado? Podemos explicar por que ele tomou certa decisão? CI/CD em MLOps, quando bem implementado, facilita muito responder a essas perguntas, pois todo o ciclo é rastreado e registrado.

Vamos discutir como a automação e as práticas de MLOps apoiam a governança de modelos e o cumprimento de regulamentações.

Auditorias e Trilhas de Auditoria: em setores como financeiro e saúde, pode ser exigido por lei manter logs de quando modelos foram atualizados e por quem foram aprovados. Uma pipeline automatizada naturalmente gera essa trilha – cada execução do pipeline é um registro com timestamp, versão de código, versão de dados e resultado (métricas, aprovação ou reprovação). Com as ferramentas adequadas, um auditor pode consultar: “Modelo versão 12 implantado em 10/02/2026 às 14:35, aprovado automaticamente após atingir 0.87 de AUC em validação, dados de treino referenciados pelo dataset commit abc123 (link), código commit def456 (link)”.

Essa transparência traz confiança e facilita certificações como, por exemplo, ISO ou requerimentos do Model Risk Management em bancos. Sem MLOps, muitas vezes esses registros ficam dispersos em e-mails, planilhas e memórias da equipe, dificultando atender a auditorias em cima da hora.

Conformidade Regulatória: leis emergentes, como o Regulamento Europeu de IA (AI Act) e, no Brasil, resoluções do Banco Central sobre model risk, impõem exigências como explicar decisões de modelos e gerenciar risco de vieses. O MLOps não resolve isso sozinho, mas é a plataforma sobre a qual soluções podem ser acopladas. Por exemplo, se precisamos garantir que um modelo de crédito não use certos atributos sensíveis, podemos incluir uma etapa automatizada no pipeline de validação que verifica a importância das features ou gera um relatório de interpretabilidade (como SHAP values) e sinaliza se há potencial viés. Esse relatório pode ser armazenado junto com o modelo no Model Registry. Além disso, um processo de CI bem estruturado permite facilmente reproduzir os resultados para

atender pedidos regulatórios – se um órgão solicitar “retreine seu modelo de 2025 com os dados originais e nos mostre que ele não era enviesado”, a empresa consegue, pois tem os dados versionados e o commit do código do modelo, podendo recriar o modelo em questão exatamente via pipeline. Isso demonstra diligência e controle de processos.

Registro de Decisões Técnicas: em MLOps, integra-se frequentemente um componente de documentação viva: cada experimento ou modelo promovido pode vir acompanhado de artefatos de documentação gerados pelo pipeline – por exemplo, gráficos de performance, explicações de features e até um breve relato (às vezes preenchido manualmente pelo cientista no momento do commit) sobre mudanças. Essas documentações, armazenadas em wikis ou no próprio registry, servem para governança interna: novas pessoas na equipe podem entender por que modelo X substituiu modelo Y, quais hipóteses foram testadas etc. É semelhante a um controle de versão de decisões de negócio/modelo. Algumas empresas adotam Model Cards – cartões padronizados com informações sobre o modelo (contexto de uso, dados, métricas, considerações éticas) – que podem ser preenchidos parcialmente de forma automatizada e exigidos no pull request que propõe um modelo para produção. Isso formaliza o processo e cria accountability: sabe-se quem revisou e aprovou o quê.

Segregação de Funções e Acessos: devemos garantir que o pipeline está protegido contra alterações não autorizadas e que há controle de acesso. Ferramentas de CI/CD e plataformas de MLOps permitem definir permissões – p. ex.: apenas determinadas pessoas ou grupos podem aprovar manualmente um deploy se requerido, ou somente a conta de serviço do CI consegue registrar modelos em produção (os indivíduos desenvolvedores não conseguem fazê-lo manualmente fora do pipeline).

Implementa-se, assim, o princípio de least privilege: ninguém deve implantar um modelo passando por fora do processo automatizado. Isso evita “atalhos” perigosos e garante que toda implantação é auditada. Em termos de compliance, esse nível de controle dá confiança de que modelos passaram pelos devidos checks.

Reprodutibilidade Forense: em casos de incidentes – imagine um modelo de crédito aprovando indevidamente centenas de empréstimos – será necessário investigar rapidamente o porquê. Com a governança do MLOps, pode-se reexecutar

exatamente o pipeline daquele modelo com os dados da época para reproduzir internamente o erro e testar correções. Isso é similar às black boxes em aviação: informação completa para análise pós-incidente. Sem esses recursos, a investigação seria demorada e possivelmente inconclusiva (“não sabemos exatamente com quais dados o modelo foi treinado naquela versão...” – algo inaceitável em um cenário regulado).

Transparência e Comunicação: MLOps também permite comunicar de forma transparente para stakeholders não técnicos. Por exemplo, equipes de negócio ou compliance podem receber notificações automáticas quando um novo modelo é implantado, juntamente com um sumário das diferenças. Isso os envolve no loop e evita surpresas. Em empresas grandes, há comitês de modelos (especialmente em bancos, comitês de modelagem, risco etc.): o pipeline pode ser configurado para enviar relatórios periódicos de performance e mudanças de modelos a esses comitês, facilitando o acompanhamento e aprovação formal.

Em conclusão, governança e compliance deixam de ser atividades paralelas e manuais para se integrar à própria esteira de MLOps. Cada passo do CI/CD contribui para construir um robusto “dossiê” dos modelos: sabemos o que, quando, como e por quem algo foi feito. Isso não só ajuda a cumprir regras e regulamentos, mas também molda uma cultura de responsabilidade: os membros da equipe sabem que suas ações (commits, configurações) ficam registradas e serão avaliadas objetivamente.

No longo prazo, esse histórico permite até análise de processo – como melhorar o pipeline, onde ocorrem mais falhas etc. De certo modo, MLOps e governança se fortalecem mutuamente: boas práticas de engenharia tornam a governança mais fácil; requisitos de governança (como auditoria) estimulam a adoção de pipelines disciplinados. Para organizações em setores regulados, estabelecer um pipeline MLOps robusto se tornou praticamente mandatório para escalar suas iniciativas de IA de forma segura.

Gestão de features flags e experimentos

Em desenvolvimento de software, o uso de feature flags (chaves de recurso) tornou-se comum para habilitar/desabilitar funcionalidades em tempo real sem reimplantar código. No contexto de MLOps, estratégias semelhantes permitem controlar gradualmente a liberar modelos ou experimentar com múltiplos modelos simultaneamente de forma segura. Vamos explorar como feature flags, testes A/B e bandits podem ser aplicados para gerenciar versões de modelos em produção e conduzir experimentos contínuos.

Feature Flags para Modelos: uma feature flag é essencialmente um interruptor que pode ativar um comportamento. Em MLOps, podemos usar flags para controlar qual modelo usar para um determinado tráfego ou segmento de usuários. Por exemplo, podemos ter uma flag `modelo_v2_enabled` que, quando ligada, faz o sistema usar o Modelo V2 e, quando desligada, reverte ao Modelo V1. Essa flag pode ser ligada apenas para uma porcentagem de usuários ou para um grupo específico (como funcionários internos, em um dogfooding).

Ferramentas especializadas (LaunchDarkly, Azure App Config, etc.) permitem gerenciamento centralizado de flags e segmentação: “ativar flag X para 10% dos usuários aleatoriamente” ou “ativar apenas para região = EUA”. Aplicado a modelos, isso é bastante poderoso – sem reimplantar nada, podemos direcionar quem recebe qual modelo. Por exemplo, se queremos testar um modelo de recomendação novo apenas no Brasil inicialmente, configuramos a flag para true apenas para `country=BR`. O código da aplicação ou do serviço de recomendação deve estar instrumentado para checar a flag e selecionar o modelo correspondente. Isso pode ser implementado via chamadas de API a um serviço de flags (que normalmente tem latência muito baixa) ou carregando um arquivo de config atualizado periodicamente.

A principal vantagem de feature flags é a reversibilidade instantânea: se o modelo novo causar problemas, desliga-se a flag e todos imediatamente voltam ao modelo antigo – semelhante a um rollback, mas centralizado e fácil de operar, inclusive fora do horário de trabalho via um painel web. Em MLOps isso se alinha ao Canary; de fato, canary strategies às vezes são implementadas internamente via flags. Além disso, flags podem segmentar por critérios de negócio: ex. ativar modelo v2

apenas para novos usuários (talvez testando se ele influencia retenção para novatos). Ou talvez se descobre que modelo v2 funciona bem para um subconjunto (ex.: para usuários com poucos dados, o modelo v1 regride mal; a solução: mantê-los no v1 e outros no v2 via flag logic). Essa mescla granular é um cenário complexo, mas real – e flags dão essa flexibilidade.

Ferramentas como LaunchDarkly inclusive oferecem experimentation e coleta de métricas integradas, facilitando analisar o impacto da flag ativada ou não. Em termos de governança, as flags e configurações podem ser consideradas parte do MLOps config e gerenciadas via IaC ou controladas com restrições de acesso (quem pode virar qual flag).

A/B Testing de Modelos: o teste A/B é um experimento controlado em produção onde se compara duas variantes (A e B) com segmentos aleatórios de usuários para avaliar diferenças de métricas. Em MLOps, essas variantes geralmente seriam “Modelo antigo” vs “Modelo novo” (ou possivelmente vs “nenhum modelo” dependendo do caso). Suponha um modelo de recomendação de produto – a métrica de sucesso pode ser taxa de conversão. Podemos fazer um A/B: grupo A de usuários vê recomendações do modelo atual, grupo B vê do modelo proposto. Após um período (dias ou semanas), comparamos as métricas (ex.: 2.5% conversão no grupo A vs 2.7% no grupo B). Se a diferença for estatisticamente significativa (usando teste t, qui-quadrado, etc., usual no meio), decide-se se B é melhor e o implementa globalmente ou descarta. O A/B testing é a abordagem científica preferencial para mudanças que impactam usuários, pois isolam o efeito da mudança.

Para integrá-lo no MLOps, precisamos de segmentação aleatória replicável (por usuário, para que a experiência do mesmo seja consistente a cada visita), coleta de métricas confiável (geralmente via analítica do produto ou logging) e ferramentas de análise. Muitas empresas usam plataformas internas ou de terceiros para gerenciar experimentos (como Optimizely X, Google Optimize etc., embora esses foquem muito em interface, é possível usar para lógicas de back-end também). O MLOps pipeline pode cooperar facilitando a alternância de modelos. Em termos práticos, A/B de modelos é semelhante a canary 50/50, mas com a ênfase no rigor estatístico.

Um cuidado: a duração e tamanho do experimento precisam ser suficientes para significância estatística. Frequentemente recorre-se a calculadoras de tamanho

amostral e testes sequenciais. Em cenários de baixa taxa de eventos, pode levar bastante tempo. Durante um A/B, convém desativar outros changes no sistema que possam confundir os resultados (princípio de experimento isolado). Ferramentas MLOps podem logar qual modelo atendeu cada usuário e com que resultado para facilitar a análise.

Ao final, se B for vencedor, rota todo tráfego para B; se não, mantém A. Um exemplo hipotético: BERT ranking model foi testado via A/B no buscador do Google antes de roll-out total, mostrando melhoria de NDCG em queries, justificando adoção global. O A/B test serviu para convencer com dados. Enquanto A/B testa com grupos fixos para tirar conclusão final, os Multi-Armed Bandits (MAB) oferecem uma forma adaptativa de alocar tráfego entre variantes, maximizando progressivamente uma métrica. A analogia é com alavancas de caça-níqueis (bandit) com retornos incertos – o algoritmo tenta equilibrar exploração (testar variantes para ganhar conhecimento) e exploração (favorecer a aparentemente melhor). Em MLOps, MABs podem ser usados para rotear tráfego dinamicamente entre múltiplos modelos para otimizar uma métrica em tempo real. Por exemplo, se estamos incertos entre o modelo A e B, um algoritmo bandit (como Thompson Sampling ou UCB1) inicialmente distribui tráfego similarmente, mas conforme observa resultados (e.g. cliques por página vista), começa a direcionar mais usuários para o modelo que está performando melhor até então.

Em tese, isso leva a maioria dos usuários a obterem a melhor experiência mesmo durante o experimento, ao custo de um pequeno percentual de exploração contínua para manter aprendizado. Ao longo do tempo, o bandit pode convergir em praticamente 100% para o melhor modelo (sendo assim uma forma adaptativa de deployment). Isso é útil quando o custo de perder performance é alto – não queremos servir modelos ruins para metade da base por semanas se já em um dia temos evidência forte que um é melhor. Bandits fazem essa otimização on-the-fly.

Em implementação, isso requer capturar recompensas (ex.: clique=1, não clique=0) e ter um serviço ou lógica de decisão rodando que atualiza as probabilidades de alocação. Às vezes, sólidos frameworks open-source como Vowpal Wabbit (com contextual bandits) ou libraries Python (scikit-optimize bandits etc.) são usados. No MLOps pipeline, bandits podem ser integrados. Exemplo: dentro do gateway ou do

código de recomendação, o sistema sorteia qual modelo usar de acordo com a distribuição de probabilidade atual e atualiza essas probabilidades após observar o reward (recompensa). Alternativamente, pode ser um componente externo que injeta configurações, como a atualização progressiva de uma feature flag no LaunchDarkly. Uma extensão disso são os contextual bandits, que utilizam características do usuário ou do contexto para decidir qual modelo tem maior probabilidade de performance para aquele caso específico. Isso transforma o processo, literalmente, em um meta-modelo que seleciona entre modelos.

O bandit é muito popular em otimização de UX e sistemas de recomendações/ads e sua adoção em MLOps significa um alto grau de automatização de experimentação: em vez de rodar vários A/B sequenciais, o bandit aprende e implementa a seleção ótimo de modelo.

Como já abordado, o rollout gradual (canary progressivo) pode ser visto como um continuum entre A/B testing e full release, com monitoramento. Configurar isso manualmente cada vez é oneroso; por isso, MLOps pode automatizar partes do rollout. Por exemplo, se nas últimas 24h a métrica de efeito do modelo B for $\geq$ a do modelo A com alta confiança, um script pode aumentar automaticamente o tráfego do modelo B de 50% para 80%. Isso seria um experimento sequencial adaptativo que vira rollout se as coisas vão bem.

Em contrapartida, Kill Switch refere-se a ter a capacidade (manual ou auto) de desligar uma funcionalidade completa imediatamente se algo der errado – no contexto de modelos, seria poder retirar o modelo novo do ar com flip de flag ou config (que já discutimos via feature flag e auto-rollback). Em operações de ML, às vezes descobrimos problemas qualitativos ou éticos tardios – ex.: modelo novo tem viés indesejado – um kill switch permite cessar seu uso instantaneamente. Este termo enfatiza a prontidão para recuar integralmente, mais forte do que um rollback típico (que pode ainda manter 5% rodando no bandit; kill switch zera 0%). Montar kill switches (via flags ou toggles de config central) é parte de resiliência do sistema e deve ser pensado de antemão. O fiasco do Microsoft Tay (chatbot) é um exemplo clássico: ele teve que ser “killed” manualmente quando usuários ensinaram condutas impróprias, mas não havia kill switch autônomo – foi uma intervenção emergencial humana. MLOps poderia prever triggers para kill. Por exemplo: detecção de outputs

tóxicos e autodesligamento, especulativo mas possivelmente real em sistemas sensíveis.

EMEND

MERCADO, CASES E TENDÊNCIAS

Organizações de diversos setores já colhem os benefícios de CI/CD aplicado a Machine Learning, tornando seus fluxos de ML mais confiáveis e escaláveis. Vejamos alguns exemplos e casos de sucesso.

Fintechs

Empresas financeiras de tecnologia adotaram MLOps para automatizar a validação e deploy de modelos de crédito e detecção de fraudes. O Nubank, por exemplo, implementou pipelines contínuos de ML que processam 50 milhões de transações diárias; com isso, conseguiram reduzir em 73% os falsos positivos em detecção de fraude ao atualizar o modelo frequentemente em produção. A solução substituiu regras manuais por aprendizado não supervisionado e, graças ao MLOps, novos padrões de fraude são incorporados quase em tempo real, economizando milhões em perdas e investigações desnecessárias. Em fintechs menores, modelos de crédito que antes levavam semanas para serem recalibrados após mudanças de mercado hoje se ajustam em dias ou horas com automação – garantindo que as decisões de empréstimo estejam sempre usando os dados mais recentes (vantagem competitiva crucial em juros dinâmicos).

Healthtechs

Na saúde, CI/CD para ML se traduz em deploy controlado de modelos clínicos. Hospitais e laboratórios empregam MLOps para validar algoritmos de diagnóstico por imagem sob rigorosos critérios antes de qualquer uso clínico. Um caso notável é o de um grande hospital que implementou pipelines MLOps para um modelo de detecção de tumores em radiografias, integrando validações automáticas de performance em diferentes subgrupos demográficos para evitar viés. Somente após passar por todas as verificações e uma revisão humana o modelo é implantado gradualmente em prática clínica. Essa abordagem contínua permitiu atualizações frequentes do modelo à medida que novos dados de pacientes chegavam, melhorando a acurácia em

detecções raras sem interromper o fluxo do hospital. Além disso, do ponto de vista regulatório, a automação registra todo o histórico de versões do modelo – facilitando auditorias pela FDA ou órgãos competentes sobre quem aprovou cada alteração e com quais resultados (compliance by design).

E-commerce

Empresas de comércio eletrônico são altamente dependentes de modelos de recomendação e precificação. Com MLOps, elas conseguem atualizar continuamente modelos de recomendação para refletir tendências de consumo quase instantaneamente. Algumas empresas tornaram-se referências ao priorizar end-to-end lifecycle management dos modelos de recomendação, integrando dados → modelo → deploy de forma contínua. Essas empresas combinam automação e colaboração multifuncional: engenheiros de ML, de dados e de produto trabalham em conjunto com pipelines integrados, encurtando ciclos de inovação e mantendo alinhamento com objetivos de negócio.

Em resultados

A Netflix credita parte do aumento de engajamento de usuários à capacidade de experimentar e refinar seus algoritmos quase diariamente através de MLOps robusto. Já uma gigante varejista utiliza MLOps para atualizar modelos de preços dinâmicos e estoques de forma segura. Qualquer ajuste de modelo de precificação passa por testes A/B automatizados (que a Amazon executa constantemente em pequenas fatias de tráfego) e só escala para todos os produtos se provar aumentar conversões sem prejudicar margens. Esse sistema automatizado de experimentação em larga escala foi fundamental para a Amazon iterar rapidamente suas estratégias de pricing a cada evento sazonal ou concorrência, com evidências empíricas e sem intervenção manual extensiva.

Indústria e iot

Em manufatura e operações industriais, MLOps viabiliza automação de modelos preditivos em produção. Um exemplo: a Siemens implementou pipelines de ML para previsão de falhas em máquinas de suas fábricas inteligentes. Sensores geram fluxos contínuos de dados e os modelos de predição de manutenção corretiva são retreinados semanalmente via um pipeline CI/CD. Com isso, a Siemens relatou reduzir paradas não planejadas em 30%, pois os modelos se mantêm atualizados com o desgaste real dos equipamentos. Além disso, eles integraram o pipeline de ML ao sistema de ordens de serviço – assim, quando o modelo (após passar por validação no pipeline) detecta alta probabilidade de falha em um motor específico, automaticamente dispara uma ordem de inspeção antes que a falha aconteça.

Esse é um exemplo de MLOps + DevOps = BizOps, unindo predição automatizada a processos operacionais. Outra tendência industrial é usar MLOps para calibrar modelos de controle de qualidade em tempo real: se um novo lote de matéria-prima apresenta propriedades ligeiramente diferentes, o modelo de controle ajusta parâmetros produtivos em minutos, e essa atualização segue todos os protocolos e log do pipeline (importante para ISO 9001 e rastreabilidade industrial).

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como as boas práticas e automação via CI/CD são fundamentais para operacionalizar Machine Learning em escala. Iniciamos entendendo os desafios únicos de levar modelos de ML do experimento à produção – como garantir qualidade de dados, reprodutibilidade e monitoramento contínuo –, e vimos que as soluções residem na aplicação disciplinada de princípios de DevOps ampliados para o contexto de ML (MLOps).

Percorremos os componentes essenciais de um pipeline MLOps completo: desde a integração contínua focada em código e dados, passando pela entrega contínua que assegura deploys seguros e auditáveis de modelos (com possibilidades de rollbacks e blue-green/canary deployment), até a importância de monitorar modelos em produção e fechar o ciclo de realimentação com retreinamentos automatizados.

Conectamos fundamentos (CI para ML, versionamento e infraestrutura) a casos reais, demonstrando que MLOps é a evolução necessária para escalar IA em qualquer organização. Discutimos como validar modelos via feature flags e testes A/B, sem negligenciar segurança e custos. O resultado é uma visão 360° para tornar o ciclo de vida de ML contínuo, incremental e mensurável.

Você agora domina os pontos críticos e as melhores práticas de MLOps, essenciais para garantir que modelos permaneçam eficazes em produção. Aplicar esses conceitos aumenta a robustez técnica e fortalece uma cultura de dados e qualidade contínua. Para consolidar o aprendizado, utilize as videoaulas com demonstrações práticas. Sucesso em suas implantações e até a próxima etapa!

REFERÊNCIAS

BRECK, E.; CAI, S.; NIELSEN, E.; SALIB, M.; SCULLEY, D. “**The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction.**” In: IEEE International Conference on Big Data, 2017. Disponível em: https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/aa_d9f93b86b7addfea4c419b9100c6cdd26cacea.pdf. Acesso em: 24 mar. 2026.

EKEN, B.; PALLEWATTA, S.; Tran, N. K.; Tosun, A.; Babar, M. A. “**A Multivocal Review of MLOps Practices, Challenges and Open Issues.**” arXiv preprint arXiv:2406.09737, 2024. Disponível em: <https://arxiv.org/abs/2406.09737>. Acesso em: 24 mar. 2026.

GOOGLE CLOUD. **MLOps: Continuous delivery and automation pipelines in machine learning.** 2024. Disponível em: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>. Acesso em: 24 mar. 2026.

HERMANN, J.; DEL BALSIO, M. “**Meet Michelangelo: Uber’s Machine Learning Platform.**” Uber Engineering Blog, 05 set. 2017. Disponível em: <https://www.uber.com/blog/michelangelo-machine-learning-platform/>. Acesso em: 24 mar. 2026.

HUYEN, C. **Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications.** 1ª Ed. Sebastopol: O’Reilly Media, 2022. ISBN: 9781098107963.

KUMARAVELU, A. “**Evolution of MLOps: Latest Trends and Innovations Shaping Machine Learning Operations.**” International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET), v.13, n.8, 2024. Disponível em: https://www.ijirset.com/upload/2024/august/17_Evolution.pdf. Acesso em: 24 mar. 2026.

LWAKATARE, L. E.; CRNKOVIC, I.; RÅNGE, E.; BOSCH, J. “**From a data science driven process to a continuous delivery process for machine learning systems.**” In: PROFES 2020 – 21st International Conference on Product-Focused Software Process Improvement, Torino, 2020. Springer, p.185–201.

RATNA, S. A. “**What top organizations implementing MLOps have in common: Real-world use cases.**” ManageEngine Blog – CXO Focus, 01 maio 2025. Disponível em: <https://www.manageengine.com/it-operations-management/cxo-focus/insights/mlops-case-studies-and-best-practices.html>. Acesso em: 24 mar. 2026.

SCULLEY, D.; HOLT, G.; GOLOVIN, D.; DAVYDOV, E.; PHILLIPS, T.; EBNER, D.; CHAUDHARY, V.; YOUNG, M.; CRESPO, J-F.; DENNISON, D. “**Hidden Technical Debt in Machine Learning Systems.**” In: Advances in Neural Information Processing Systems 28 (NIPS 2015). Montréal, 2015. Disponível em:

<https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>. Acesso em: 24 mar. 2026.

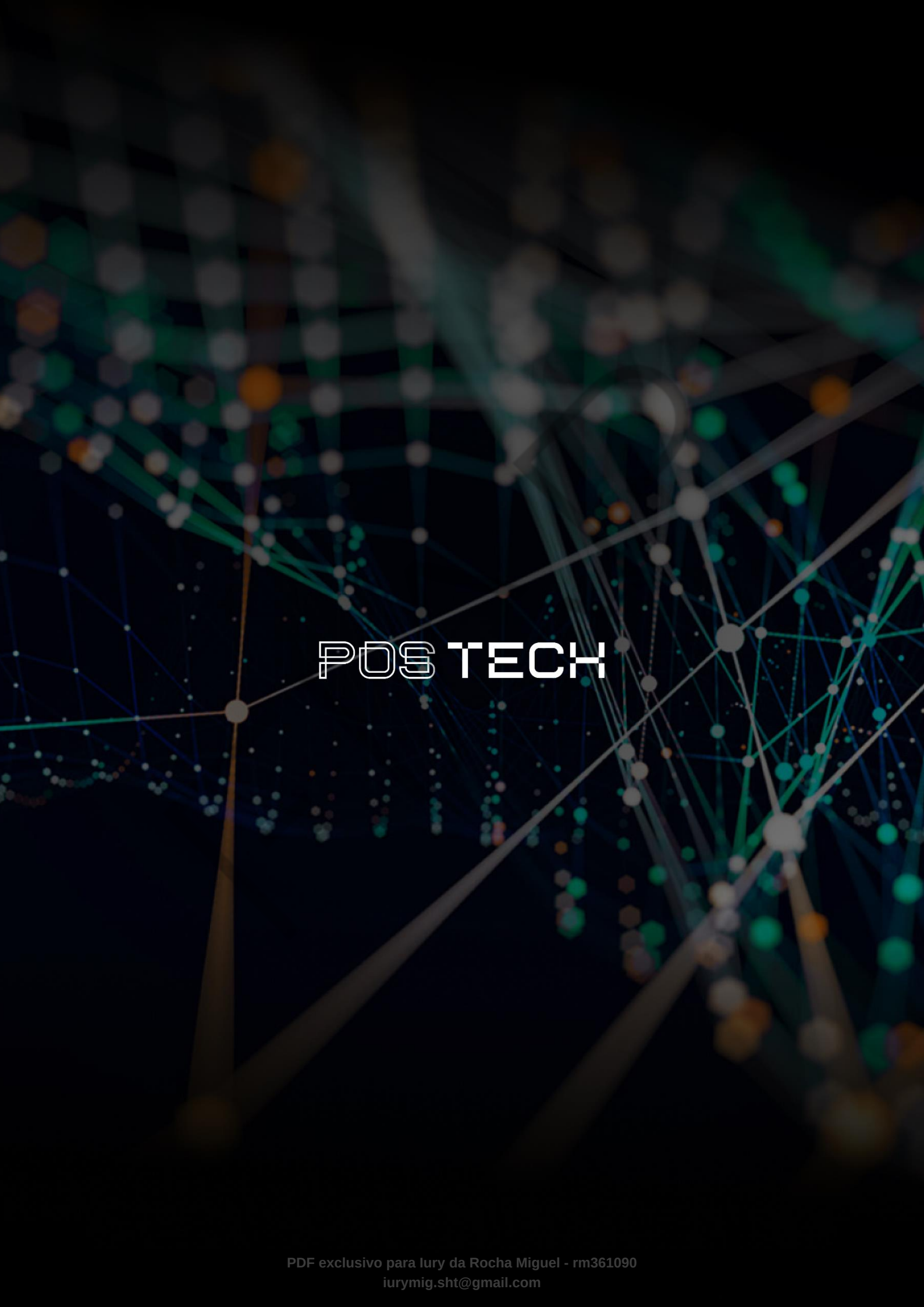
WINDHEUSER, C.; SATO, D.; SADOVSKY, A.; COOPERBERG, D.; WILLIS, G.; PODUSKA, J.; FALGOUT, J.; TONG, D. **MLOps: Continuous Delivery for Machine Learning on AWS**. Whitepaper, AWS & ThoughtWorks, 2020. Disponível em: <https://d1.awsstatic.com/whitepapers/mlops-continuous-delivery-machine-learning-on-aws.pdf>. Acesso em: 24 mar. 2026.

EMEND

PALAVRAS-CHAVE

CI/CD. MLOps. Automação.

EMENDAS



POSTECH